Transferência de arquivos P2P

Sistemas Distribuídos - TP2 | CEFET-MG | 2026/2
Professora: Michelle Hanne.
Integrantes: Augusto Coelho Fagundes e ADJAILSON FREIRE DE SÁ FILHO.
30/10/2026.

Código-fonte: https://github.com/Augusto42/sd-tp2-p2p-2026

1. Arquitetura e decisões

Cada peer é um processo Python com um servidor TCP e tarefas clientes no mesmo event loop asyncio. Ele atende solicitações enquanto baixa de seus vizinhos. A biblioteca padrão reduz dependências e permite concentrar o trabalho em protocolo, concorrência e integridade.

Os vizinhos são estáticos, configurados por IP e porta. Não foi usado tracker, pois é opcional. A matriz usa malha completa: com quatro peers, cada um configura os outros três como vizinhos. Cada leecher estabelece contato com todos, mesmo se um download curto concluir antes da conexão com um vizinho que iniciou depois.

O arquivo é dividido logicamente em blocos de 1.024 ou 4.096 bytes. O metadado JSON informa versão, tamanho, tamanho de bloco, nome informativo, hashes por bloco e hash completo. Todos recebem esse metadado previamente; ele é considerado confiável.

Os blocos ficam em um arquivo parcial em disco, gravados por offset, evitando carregar todo o arquivo em memória. O conjunto have registra os blocos verificados; busy reserva pedidos em voo e evita que duas tarefas solicitem o mesmo bloco ao mesmo tempo.

Após cada bloco passar na checagem de tamanho e SHA-256, ele fica disponível para upload. Ao concluir, verifica-se o SHA-256 completo e renomeia-se o parcial para arquivo_final.bin. O peer continua servindo até receber SIGINT ou SIGTERM.

2. Protocolo e implementação

TCP fornece um fluxo de bytes, não mensagens. Por isso cada frame usa prefixo de comprimento de 4 bytes big-endian e corpo de até 8 MiB. readexactly trata frames divididos em múltiplas leituras. Conexões persistentes evitam reconectar a cada bloco.

Mensagem	Conteúdo / função
have (JSON)	Pedir lista dos índices disponíveis
get (JSON)	Pedir índice de bloco e identificar arquivo pelo hash
J + JSON	Responder inventário de blocos
B + bytes	Entregar bloco binário sem base64
E + texto	Indicar arquivo incorreto ou bloco indisponível

Um pedido fica em voo por conexão, dispensando identificador de correlação. Cada vizinho possui sua própria tarefa de download. O servidor mantém uma tarefa por conexão recebida. Gravação/atualização do conjunto e leitura do bloco não contêm await; assim, outras tarefas não alteram o ponteiro compartilhado entre seek e read/write.

Conexão tem timeout de 2 s; resposta, 5 s; servidor ocioso, 15 s. Falhas liberam reservas e tentam novamente após 100 ms; falta de blocos novos gera nova consulta após 20 ms. Estes valores privilegiam uma demonstração em rede local. Não há limite global de espera.

Hashes por bloco rejeitam dados corrompidos antes de publicá-los. O hash final confirma remontagem. O nome do metadado é informativo: a saída usa nome local fixo, evitando usar dados remotos como caminho de escrita.

3. Procedimento experimental

Executaram-se 24 combinações: 2/4 peers x 1.024/4.096 bytes x 10/20 KiB, 1/5/10/20 MiB. KB e MB do enunciado foram interpretados como unidades binárias. Cada caso usa conteúdo aleatório, processos independentes e sockets TCP reais em loopback.

Cada transferência verifica tamanho, SHA-256 e origem dos blocos nos logs. Os testes preservam metadados, eventos JSONL, verificações e tabelas CSV/JSON; os binários temporários são descartados. A repetição gera novos dados. Cada configuração teve uma execução; não há intervalos de confiança.

4. Resultados da matriz completa

Arquivo	2 peers / 1 KiB	2 peers / 4 KiB	4 peers / 1 KiB	4 peers / 4 KiB
10 KiB	0.2859 s	0.2866 s	0.6833 s	0.5333 s
20 KiB	0.3864 s	0.2889 s	0.6016 s	0.6040 s
1 MiB	0.6530 s	0.5120 s	0.8014 s	0.6138 s
5 MiB	1.1169 s	0.7149 s	2.1482 s	1.5801 s
10 MiB	1.8231 s	0.8680 s	6.9813 s	1.2639 s
20 MiB	3.1995 s	1.5319 s	10.2469 s	5.0194 s

Integridade: 24/24 combinações aprovadas. Todos os arquivos finais tiveram tamanho e SHA-256 iguais ao original. Os logs confirmam que cada leecher se conectou a todos os vizinhos configurados. Cada bloco foi recebido uma única vez e sua fonte pertence à lista de vizinhos.

Os valores acima são medições da malha completa. Quatro peers entregam três cópias em paralelo, com possibilidade de redistribuição entre leechers. Os tempos incluem inicialização, cópia da semente, espera pelo contato com todos os vizinhos, polling e logs; não medem apenas transmissão de bytes.

O campo aggregate_mib_s do CSV é tamanho x (peers - 1) / tempo / 2^20: taxa agregada de cópias entregues, não capacidade física de enlace. Rede local em loopback, caches e escalonamento limitam qualquer generalização para LAN/WAN.

Ambiente detalhado em resultados/ambiente.json. Logs por caso em resultados/<caso>/peer_N.jsonl; verificacao.json associa hashes, fontes, tempo e contagem de repasses parciais.

5. Casos adicionais e avaliação

Compartilhamento parcial: integridade aprovada em 0.7941 s; 43 envios por leechers antes de completar; 6 novas tentativas.

Seeder tardio: integridade aprovada em 1.1090 s; 0 envios por leechers antes de completar; 7 novas tentativas.

Malha completa: integridade aprovada em 0.9637 s; 98 envios por leechers antes de completar; 13 novas tentativas.

O cenário de compartilhamento parcial usa atraso artificial de 1 ms; no seeder tardio, a fonte inicia 700 ms depois.

Queda temporária: quatro peers em malha, uma semente. SIGKILL interrompe a semente com downloads ainda incompletos. Os leechers registram falhas, mantêm os blocos válidos e reconectam após seu retorno. Todos concluem com tamanho e SHA-256 corretos, sem baixar novamente blocos já recebidos.

Queda permanente: quatro peers em malha, duas sementes iniciais. Uma é morta durante a transferência. Os três processos restantes concluem sem seu retorno, recebendo novos blocos de fontes alternativas. A redundância garante disponibilidade dos blocos; não é possível reconstruir blocos que só existiam em um peer definitivamente perdido.

Testes unitários: 11/11 aprovados. Cobrem último bloco parcial e montagem fora de ordem, corrupção, tamanho incorreto, arquivo vazio, metadado inválido, tamanho de bloco inválido, hash final divergente, frame fragmentado, frame truncado limite de frame e conexão a vizinho mesmo após download curto concluir.

6. Limitações e conclusão

A solução atende à simetria, fragmentação, registro local de blocos, redistribuição durante o download e montagem verificada. Usa operações de disco síncronas curtas no event loop; a aplicação não garante durabilidade contra queda de energia. Não implementa autenticação/TLS, rarest-first, descoberta de peers, retomada persistente nem distribuição automática de metadados. Vizinhos são cooperativos e o metadado é confiável.

As 29 integrações e os 11 testes unitários passaram. Melhorias futuras: retomada com bitmap persistente, inventários incrementais, agendamento de blocos e ensaios repetidos em máquinas/redes distintas.

Referências: enunciado SD_TP2_2_2026.pdf, seções 1 a 3, pp. 1-2; código e evidências no repositório acima. Instruções reproduzíveis no README e especificação detalhada em docs/protocolo.md.